

PSI3441 - Arquitetura de Sistemas Embarcados

Atividade 1

Gabriel Ivo Bozi

14571845

5 de maio de 2026

1 POR QUE OS *LEDS* SÃO *ACTIVE LOW* (ACENDEM QUANDO SE COLOCA 0 NA SAÍDA)?

Na eletrônica dos microcontroladores, os pinos de saída geralmente possuem uma capacidade consideravelmente maior de drenar corrente para o terra (*current sink*) do que de fornecer corrente a partir da fonte de alimentação (*current source*).

Portanto ligar os *leds* na configuração *Active Low* reduz o estresse elétrico no *chip* e previne danos físicos às portas do microcontrolador. Além disso, essa configuração garante estabilidade, evitando que o *led* pisque ou acenda acidentalmente devido a ruídos no barramento durante o processo de inicialização do sistema, momento em que os pinos frequentemente assumem um estado de alta impedância.

2 QUAIS FUNÇÕES VOCÊ USOU PARA ACENDER E APAGAR OS *LEDS*?

Para configuração e verificação de como os *leds* devem se comportar foram utilizadas as seguintes funções:

`gpio_pin_configure_dt()` Configura os pinos dos *leds* como saída e define o estado padrão.

`gpio_is_ready_dt()` Utilizada como etapa de validação de segurança para confirmar se o periférico mapeado está pronto para receber comandos do sistema.

Para acionamento dos *leds* foram utilizadas as seguintes funções:

`gpio_pin_toggle()` Inverte o nível lógico no pino.

`gpio_pin_set_dt()` Utilizada para acender e apagar os *leds* de fato, alterando o estado lógico do pino (o valor passado como parâmetro representa o estado lógico desejado para o pino, sendo interpretado conforme a configuração do hardware, por exemplo, *Active Low* ou *Active High*)..

3 EXPLIQUE O QUE É O *DEVICE TREE*

O *DeviceTree* é uma estrutura de dados que descreve o *hardware* da plataforma de forma declarativa. Em vez de utilizar endereços ou identificadores fixos no código, ele permite referenciar periféricos por meio de aliases e propriedades, com isso um pino com uma função específica, por exemplo o *led*, pode funcionar em diferentes plataformas com arranjos dos pinos diferentes. , e é capaz de descrever propriedades dos pinos, por exemplo a polaridade do *led* já que nessa placa ele é ativo em nível lógico baixo. Além de ser capaz de fazer verificações de segurança e portabilidade em tempo de compilação.

4 EXPLIQUE AS ABSTRAÇÕES FEITAS PELO SISTEMA OPERACIONAL

Sistemas operacionais de tempo real como o *Zephyr* constroem camadas de abstração de hardware para isolar o desenvolvedor da manipulação direta de registradores e mapeamento da memória por meio do *linker*.

Anexos

1 CÓDIGO MAIN.C UTILIZADO

```
#include <zephyr/kernel.h>
#include <zephyr/device.h>
#include <zephyr/drivers/gpio.h>

#define SLEEP_TIME_MS 500

// Define os LEDs usando Device Tree Alias
#define LED0_NODE DT_ALIAS(led0) // green
#define LED1_NODE DT_ALIAS(led1) // blue
#define LED2_NODE DT_ALIAS(led2) // red

#if DT_NODE_HAS_STATUS(LED0_NODE, okay) || DT_NODE_HAS_STATUS(LED1_NODE, okay) ||
    DT_NODE_HAS_STATUS(LED2_NODE, okay)
    static const struct gpio_dt_spec led0 = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
    static const struct gpio_dt_spec led1 = GPIO_DT_SPEC_GET(LED1_NODE, gpios);
    static const struct gpio_dt_spec led2 = GPIO_DT_SPEC_GET(LED2_NODE, gpios);
#else
    #error "Unsupported board: leds device tree alias is not defined"
#endif

// Definição dos estados possíveis para a máquina de estados
// red
// yellow = green + red
// green
typedef enum
{
    STATE_RED,
    STATE_YELLOW,
    STATE_GREEN
} led_state_t;

void main()
{
    // Verifica se os devices estão prontos
    if (!gpio_is_ready_dt(&led0) || !gpio_is_ready_dt(&led1) || !gpio_is_ready_dt(&led2))
    {
        printk("Error: One or more leds devices are not ready\n");
        return;
    }

    // Configura os 3 pinos como saída e inicia desligados
    gpio_pin_configure_dt(&led0, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&led1, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&led2, GPIO_OUTPUT_INACTIVE);

    printk("leds ready to blink using a state machine\n");

    // Inicializa a máquina de estados no verde
    led_state_t current_state = STATE_GREEN;

    while (1)
    {
        // Desliga todos os leds
```

```

    gpio_pin_set_dt(&led0, 0);
    gpio_pin_set_dt(&led1, 0);
    gpio_pin_set_dt(&led2, 0);

    switch (current_state)
    {
        //
        case STATE_GREEN:
            gpio_pin_set_dt(&led0, 1);
            current_state = STATE_YELLOW;
            break;

        case STATE_YELLOW:
            gpio_pin_set_dt(&led0, 1);
            gpio_pin_set_dt(&led2, 1);
            current_state = STATE_RED;
            break;

        case STATE_RED:
            gpio_pin_set_dt(&led2, 1);
            current_state = STATE_GREEN;
            break;
    }

    k_msleep(SLEEP_TIME_MS);
}

return;
}

```

2 LINK DO REPOSITÓRIO

https://github.com/KryptonPlusPlus/PSI3441/tree/main/atividade_01